

Jul 09, 2026

FPGA-Based Emulation Framework for Two-Qubit Quantum Computation: Design, Implementation, and Validation

DOI

<https://dx.doi.org/10.17504/protocols.io.dm6gp4bqpgzp/v1>

Arfeen Zahra¹, Madiha Khalid¹, Naveed Riaz², Salman Waheed³, Umar Mujahid⁴, Muhammad Najam ul Islam⁴

¹Department of Computing, School of Electrical Engineering and Computer Science, National University of Sciences and Technology (NUST), Islamabad, Pakistan;

²Department of Electrical and Computer Engineering, International Islamic University, Pakistan;

³Department of Information Technology, Georgia Gwinnett College, Georgia, United States of America;

⁴Department of Electrical Engineering, Namal University, Mianwali, Pakistan

National university of sc...



Arfeen Zahra

NUST

Create & collaborate more with a free account

Edit and publish protocols, collaborate in communities, share insights through comments, and track progress with run records.

Create free account

OPEN  ACCESS



DOI: <https://dx.doi.org/10.17504/protocols.io.dm6gp4bqpgzp/v1>



Protocol Citation: Arfeen Zahra, Madiha Khalid, Naveed Riaz, Salman Waheed, Umar Mujahid, Muhammad Najam ul Islam 2026. FPGA-Based Emulation Framework for Two-Qubit Quantum Computation: Design, Implementation, and Validation. **protocols.io** <https://dx.doi.org/10.17504/protocols.io.dm6gp4bqpgzp/v1>

License: This is an open access protocol distributed under the terms of the [Creative Commons Attribution License](#), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited

Protocol status: Working

We use this protocol and it's working

Created: July 09, 2026

Last Modified: July 09, 2026

Protocol Integer ID: 320637

Keywords: Bell States, Deutsch Algorithm, FPGA, Grover's search, IBM AER Simulator, Quantum Algorithms, QKD, Resource-efficient Quantum Emulator, Superdense coding

Disclaimer

The validation methodology described here was implemented and verified using Xilinx Vivado 2018.2 (XSim). While the underlying approach is simulator-agnostic in principle, users replicating this work on other Vivado versions or alternative Verilog simulators (e.g., ModelSim, Icarus Verilog) should note that exact reproduction may require minor adaptation — for instance, re-customization of IP cores if version-specific IP is used, or adjustment of simulator-specific commands and file paths. The core RTL and testbench logic, however, are standard Verilog and are expected to be portable across environments with minimal modification.

Abstract

Quantum computing leverages quantum mechanics to perform complex computations efficiently, yet its practical implementation is hindered by hardware-software interdependence and quantum state instability. This paper advances lightweight FPGA-based quantum emulation by incorporating a universal quantum gate set, including parameterized Phase gates and the CNOT gate, to support comprehensive quantum protocol emulation on FPGA hardware. The FPGA platform used for synthesis is Atrix 7. The proposed emulator is validated through simulations of quantum circuits, including Bell States, the Deutsch algorithm, Superdense coding, Grover's search, and BB84 QKD, with results benchmarked against IBM's noiseless quantum simulator. Utilizing FPGA's customizable parallelism and energy efficiency, this framework offers a resource-efficient, reliable platform for developing and testing quantum algorithms.

Attachments



FPGA_Quantum_Emulati.



598KB

Materials

Item | Specification

---|---

Target FPGA device | Xilinx Artix-7, xc7a200tfbg676-2 (133,800 LUTs; 267,600 flip-flops; 740 DSP blocks; 400 I/O pins; 365 × 36 KB BRAMs)

Design/synthesis tool | Xilinx Vivado Design Suite, Verilog HDL

Host workstation | PC capable of running Vivado for synthesis, implementation, and behavioural simulation

Comparison simulator | IBM Qiskit Aer (noiseless simulator)

Comparison hardware (optional) | IBM quantum hardware access, e.g. ibm_aachen, ibm_fez (via IBM Quantum cloud)

Numeric representation | Signed fixed-point Q(2,6) for standard runs; Q(2,30) for the extended-precision cost/accuracy comparison

Abstract

1 Quantum computing leverages quantum mechanics to perform complex computations efficiently, yet its practical implementation is hindered by hardware-software interdependence and quantum state instability. This paper advances lightweight FPGA-based quantum emulation by incorporating a universal quantum gate set, including parameterized Phase gates and the CNOT gate, to support comprehensive quantum protocol emulation on FPGA hardware. The FPGA platform used for synthesis is Atrix 7. The proposed emulator is validated through simulations of quantum circuits, including Bell States, the Deutsch algorithm, Superdense coding, Grover's search, and BB84 QKD, with results benchmarked against IBM's noiseless quantum simulator. Utilizing FPGA's customizable parallelism and energy efficiency, this framework offers a resource-efficient, reliable platform for developing and testing quantum algorithms.

Keywords: Bell States, Deutsch Algorithm, FPGA, Grover's search, IBM AER Simulator, Quantum Algorithms, QKD, Resource-efficient Quantum Emulator, Superdense coding.

Background

2 Quantum algorithm design has matured considerably since Deutsch's, Shor's, and Grover's algorithms, but the construction of practical, low-cost platforms for testing such algorithms remains constrained by hardware-software interdependence and the difficulty of maintaining stable quantum states. FPGA-based emulation offers a resource-efficient, deterministic alternative to cloud-based quantum hardware and CPU-bound simulators, at the cost of representing quantum information in fixed-point rather than continuous form. This protocol extends a previously published single-qubit, single-gate-set FPGA emulator (implemented on a Spartan-3 platform) to a two-qubit universal gate set implemented on a larger Artix-7 device. The universal set is completed by adding parameterized Phase, S, and T gates and an entangling CNOT gate to the existing Pauli-X, Pauli-Y, Pauli-Z, and Hadamard gates. The resulting emulator is validated against IBM's noiseless Qiskit Aer simulator and, where noted, against IBM quantum hardware (ibm_aachen, ibm_fez).

Materials and Equipment

3

Item	Specification
Target FPGA device	Xilinx Artix-7, xc7a200tfbg676-2 (133,800 LUTs; 267,600 flip-flops; 740 DSP blocks; 400 I/O pins; 365 × 36 KB BRAMs)
Design/synthesis tool	Xilinx Vivado Design Suite, Verilog HDL
Host workstation	PC capable of running Vivado for synthesis, implementation, and behavioural simulation
Comparison simulator	IBM Qiskit Aer (noiseless simulator)
Comparison hardware (optional)	IBM quantum hardware access, e.g. ibm_aachen, ibm_fez (via IBM Quantum cloud)
Numeric representation	Signed fixed-point Q(2,6) for standard runs; Q(2,30) for the extended-precision cost/accuracy comparison

Procedure

4 Part A — Build the emulator hardware architecture

Step 1. Design the quantum memory block.

Implement an n -qubit register file ($n \in \{1, 2\}$) that stores each probability amplitude as a

pair of signed Q(2,6) numbers representing the real (α) and imaginary (β) components. For a single qubit, store the two basis amplitudes (C_0, C_1) as four Q(2,6) registers. For the two-qubit extension, store the four basis amplitudes ($C_{00}, C_{01}, C_{10}, C_{11}$) as eight Q(2,6) registers (a $2^3 \times 8$ register file).

Include a write-enable flag driving an $(n+1) \times 2^{n+1}$ decoder and a $2^{n+1} \times 1$ multiplexer to control sequential read/write access.

Step 2. Design the measurement block.

Implement a Permutation-based Shuffling (PbS) function that generates an 8-bit fixed-point

pseudo-random number (PRNG) in the interval $[0,1]$ for each measurement shot.

For single-qubit measurement, assign the outcome $|0\rangle$ if the PRNG falls in $[0,$

$|C_0|^2)$, otherwise assign $|1\rangle$. For two-qubit measurement, partition $[0,1]$ into

four subintervals proportional to $|C_{00}|^2$, $|C_{01}|^2$, $|C_{10}|^2$, and $|C_{11}|^2$, and

assign the outcome corresponding to whichever subinterval contains the PRNG

value. Configure each measurement experiment to run for 100 shots to match the shot-based reporting convention used by IBM quantum platforms.

Step 3. Implement the single-qubit Phase, S, and T gate emulation modules.

Build the Phase gate ($P(\theta)$) module using the algebraically simplified rotation form that leaves $|0\rangle$ unchanged and rotates $|1\rangle$ by angle θ about the z-axis. Derive the S

gate as the special case $\theta = \pi/2$ and the T gate as the special case $\theta = \pi/4$,

each implemented as a separate module. For validation during this step,

initialise the input state to $|+\rangle$ and, for the Phase gate, set $\theta = \pi/3$; confirm

the output magnitudes against the theoretical prediction before proceeding.

Step 4. Implement the CNOT gate (two-qubit entangling gate) emulation module.

Build a tensor-product module that combines the control and target qubit states, then

apply the 4×4 CNOT permutation matrix using the algebraic simplification of the

complex arithmetic (rather than a literal matrix multiplication) to reduce

resource usage. Do not use Block RAM (BRAM) for any of the gate or memory

modules in this step, since the small state space of a 1- or 2-qubit system is

more efficiently handled with register-based, single-cycle-access storage.

Step 5. Implement the finite state machine (FSM) controller.

Design an FSM that, on a start signal, selects the required gate (Phase, S, T, or CNOT) for

the current circuit step, applies it to the qubit state fetched from the memory

block, writes the transformed state back to memory (or forwards it to the

measurement block), and proceeds until reaching a finish state. Instantiate

each gate as a separate reusable module so the FSM can sequence arbitrary

combinations of gates for a given circuit.

Step 6. Validate arbitrary single-qubit gate emulation via Euler (Z-X-Z) decomposition.

Confirm the emulator can realise any single-qubit unitary $U(\theta, \phi, \lambda)$ by decomposing it as $U =$

$P(\phi) \cdot H \cdot P(\theta) \cdot H \cdot P(\lambda)$, instantiating Phase and Hadamard gates in that order. As a

test case, apply this sequence to input state $|0\rangle$ using angles $\theta = \pi/3$, $\phi =$

$\pi/2$, $\lambda = \pi/4$ and compare the resulting output state against the theoretical

prediction.

Step 7. Synthesize the design and record baseline resource utilisation.

Synthesize and implement the completed architecture in Vivado targeting the Artix-7

(xc7a200tfbg676-2) device. For each gate module (Phase, S, T, CNOT), record LUT

count, flip-flop count, DSP block count, I/O pin count, BRAM count, on-chip power, critical-path latency (in clock cycles and nanoseconds), and maximum clock frequency (F_{max}).

Part B — Emulate and validate the five benchmark quantum circuits

Step 8. Bell state generation.

Construct the four Bell-state circuits ($|\Phi^+\rangle$, $|\Psi^+\rangle$, $|\Phi^-\rangle$, $|\Psi^-\rangle$) by applying a Hadamard gate

to the first qubit followed by a CNOT gate entangling it with the second qubit (with additional Pauli corrections for the $-$ and Ψ variants). Route the output of each circuit to the quantum memory block and then to the measurement block. Run each Bell-state circuit and record the measured probability distribution.

Step 9. Deutsch's algorithm.

Implement Deutsch's algorithm using the phase oracle for the XNOR function (a balanced

function) as the test case. Confirm that the measurement result of the least significant qubit is predominantly 1, consistent with a balanced function, and record the measured probability distribution for comparison with the theoretical result.

Step 10. Superdense coding.

Prepare the Bell state $|\Phi^+\rangle$ shared between a sender and receiver defined within a single

unified FSM. To transmit classical bits $(c,d) = (1,1)$, apply the conditional Pauli correction X^1Z^1 to the sender's qubit, then perform Bell-basis measurement (CNOT followed by Hadamard) at the receiver. Confirm the classical output reads predominantly '11' and record the measured probability distribution.

Step 11. Grover's search algorithm (2-qubit).

Implement the 2-qubit Grover circuit consisting of a phase oracle (marking state $|01\rangle$) and a

diffuser, applying the H-CNOT-H sequence (equivalent to a controlled-Z operation) at the behavioural level within the FSM. Run the circuit for a single Grover iteration (as required for a 2-qubit search space) and confirm the marked state $|01\rangle$ is returned with high probability.

Step 12. BB84 quantum key distribution.

Implement separate sender and receiver modules (BB84 is a communication protocol rather

than a single circuit). At the sender, randomly generate a classical key and a sequence of Z/Y measurement bases, and encode the key onto qubits accordingly. At the receiver, measure each incoming qubit against an independently and randomly chosen Z or Y basis; for the Y-basis measurement, apply a Phase gate with $\theta = 270^\circ$ followed by a Hadamard gate to replicate Y-basis measurement using the Z-basis hardware already implemented. Run the protocol for 100

key-transmission instances and record the key agreement rate between sender and receiver.

Part C — Quantify accuracy, resource cost, and cross-platform benchmarking

Step 13. Compute per-gate error rates at Q(2,6) and Q(2,30) precision.

For each unitary gate (Phase, S, T, CNOT), compute the magnitude of the theoretical and emulated output amplitudes, take the absolute deviation between them, and express this as a percentage of the theoretical magnitude. Repeat the full set of gate emulations using an extended Q(2,30) fixed-point representation and record the resulting change in error rate alongside the corresponding increase in LUT and slice utilisation.

Step 14. Benchmark accuracy and execution time against IBM Aer and IBM quantum hardware.

Using a comparison circuit consisting of qubit initialisation, a single unitary transformation, and a 100-shot measurement, run the same operation on: (a) the FPGA emulator, (b) the IBM Qiskit Aer simulator, and (c) IBM quantum hardware (e.g. `ibm_fez` for gate-level comparison, `ibm_aachen` for the Bell-state/Deutsch comparison). Record accuracy and execution time for each platform, noting that IBM hardware results additionally reflect physical noise sources (decoherence, gate error, readout error) that do not apply to the emulator or Aer.

Step 15. Benchmark the five validation circuits across all three platforms.

Execute the Bell-state circuits, Deutsch's algorithm (XNOR oracle), superdense coding, and

Grover's search on the FPGA emulator, IBM Aer, and IBM quantum hardware. For each platform and circuit, record the error rate relative to the theoretical ground truth together with FPGA resource utilisation (LUT count, slice count) where applicable.

Step 16. Perform gate-depth error accumulation analysis.

Apply a single-qubit Phase gate with $\theta = \pi/4$ repeatedly (eight consecutive applications) to accumulate a full 2π rotation, recording the emulator output and theoretical output after each application. Confirm that truncation error accumulates approximately linearly with circuit depth, consistent with the Q(2,6) format's fractional resolution of 2^{-6} .

Step 17. Document scalability limits.

Note that the register-based memory architecture scales as $O(2^W)$ with qubit width W , and

record the practical width/depth limits demonstrated ($W = 2$, $D = 6$ for the validated Deutsch instance) as the basis for any future scale-up (e.g. BRAM-based storage, multi-FPGA partitioning, or hybrid FPGA-GPU acceleration).

Expected Results

- 5 Following this protocol should reproduce approximately: 1% error for S and T gate $|0\rangle$ outcomes and 1–8% error for $|1\rangle$ outcomes at Q(2,6); a CNOT gate error of approximately 3.1% at Q(2,6), improving to approximately 2.15% at Q(2,30); an average Bell-state error of approximately 2%, a Deutsch (XNOR) error of approximately 14%, a superdense coding error of approximately 4%, and a Grover's search error of approximately 7%, each broadly comparable to IBM Aer under the same conditions; and a BB84 key agreement rate of approximately 76.75%, comparable to the theoretical 75% rate expected from basis reconciliation. On resource cost, the CNOT gate is expected to require noticeably more LUTs and slices than the single-qubit gates, and higher-precision Q(2,30) modules are expected to require more resources than their Q(2,6) counterparts.

Notes

- 6 All gate and memory modules should be implemented without BRAM; register-based storage is sufficient and more efficient at this circuit scale.
Measurement-related error is expected to be more sensitive to shot count than unitary-transformation error, since the latter depends primarily on fixed-point precision rather than sampling statistics.
Comparisons against IBM quantum hardware should be treated as functional benchmarking only, not a like-for-like hardware performance comparison, since the FPGA emulator, Aer, and IBM hardware represent fundamentally different execution environments.

Troubleshooting

- 7 If simulation mismatches occur, verify fixed-point scaling, FSM sequencing, and gate interconnections.

Protocol references

- [1] A manufacturable platform for photonic quantum computing," *Nature*, vol. 641, no. 8064, pp. 876–883, 2025.
- [2] D. Loss and D. P. DiVincenzo, "Quantum computation with quantum dots," *Physical Review A*, vol. 57, no. 1, p. 120, 1998.
- [3] S. Bravyi, O. Dial, J. M. Gambetta, D. Gil, and Z. Nazario, "The future of quantum computing with superconducting qubits," *Journal of Applied Physics*, vol. 132, no. 16, 2022.
- [4] G. Burkard, T. D. Ladd, A. Pan, J. M. Nichol, and J. R. Petta, "Semiconductor spin qubits," *Reviews of Modern Physics*, vol. 95, no. 2, p. 025003, 2023.
- [5] J. I. Cirac and P. Zoller, "Quantum computations with cold trapped ions," *Physical Review Letters*, vol. 74, no. 20, p. 4901, 1995.
- [6] Q. Computing, "Top applications of quantum computing for machine learning," 2023.
- [7] Yole Group, "Quantum technologies: The market is growing," June 2021.
- [8] A. H. Hussain, M. N. Hasan, N. U. Prince, M. M. Islam, S. Islam, and S. K. Hasan, "Enhancing cyber security using quantum computing and artificial intelligence," *PLOS ONE*, vol. 15, pp. 123–130, 2021.
- [9] Y. Moawad, "Architectures and optimisations for FPGA-based simulation of quantum circuits," PhD thesis, Glasgow, Scotland, 2025.
- [10] M. Khalid, U. Mujahid, A. Jafri, H. Choi, and N. u. I. Muhammad, "An FPGA-based hardware abstraction of quantum computing systems," *Journal of Computational Electronics*, vol. 20, no. 5, pp. 2001–2018, 2021.
- [11] P. Kaye, R. Laflamme, and M. Mosca, "An introduction to quantum computing," 2007.
- [12] A. Giorgio, "Project and implementation of a quantum logic gate emulator on FPGA using a model-based design approach," *IEEE Access*, vol. 12, pp. 41317–41353, 2024.
- [13] C. Conti, D. Volpe, M. Graziano, M. Zamboni, and G. Turvani, "Amaretto: Enabling efficient quantum algorithm emulation on low-tier FPGAs," *arXiv preprint arXiv:2411.09320*, 2024.
- [14] L. Lagostina, D. Volpe, M. Zamboni, and G. Turvani, "Aequam: Accelerating quantum algorithm validation through FPGA-based emulation," *IEEE Access*, 2025.
- [15] N. Nedjah, S. Raposo, and L. de Macedo Mourelle, "Dedicated hardware design for efficient quantum computations using classical logic gates," *The Journal of Supercomputing*, vol. 80, no. 5, pp. 7028–7070, 2024.

- [16] S. Choi and W. Lee, "Developing a Grover's quantum algorithm emulator on standalone FPGAs: optimization and implementation," *Quantum Computing Journal*, vol. 15, pp. 123–130, 2024.
- [17] E. El-Araby, N. Mahmud, M. J. Jeng, et al., "Towards complete and scalable emulation of quantum algorithms using FPGA-based architectures," *IEEE Transactions on Computers*, vol. 72, no. 8, pp. 2350–2364, 2023.
- [18] H. M. Waidyasooriya, H. Oshiyama, Y. Kurebayashi, M. Hariyama, and M. Ohzeki, "A scalable emulator for quantum Fourier transform using multiple FPGAs with high-bandwidth memory," *IEEE Access*, vol. 10, pp. 65103–65117, 2022.
- [19] N. Mahmud, B. Haase-Divine, A. Kuhnke, A. Rai, A. MacGillivray, and E. El-Araby, "Efficient computation techniques and hardware architectures for unitary transformations in support of quantum algorithm emulation," *Journal of Signal Processing Systems*, vol. 92, no. 9, pp. 1017–1037, 2020.
- [20] U. Mujahid, M. Khalid, and M. Najam-ul-Islam, "FPGA based emulation of BB92 QKD protocol," in *2023 57th Annual Conference on Information Sciences and Systems (CISS)*, 2023, pp. 1–5.
- [21] IBM Quantum, "Compute resources — ibm_aachen QPU," 2025.
- [22] IBM, "IBM Fez quantum processing unit," IBM Quantum Platform, 2026.
- [23] J. Pilch and J. Dlugopolski, "An FPGA-based real quantum computer emulator," *Journal of Computational Electronics*, vol. 18, no. 1, pp. 329–342, 2019.
- [24] I. Belfiore and A. Lee, "A scalable FPGA architecture for quantum computing simulation," *arXiv preprint arXiv:2407.06415*, 2024.